

# Hiranandani Dashboard — Production Readiness & Scope Plan

Full-proof plan so the site does not break under real, concurrent, multi-staff daily use — plus a list of dashboard features to deprioritize for now.

Prepared 2026-07-31 · Based on a direct code review of the current repository (SessionReports.tsx, /api/activity, /api/controls, auth/session/users libraries)

## 1. What the app actually looks like today

Before planning for scale, here is the honest current state, found directly in the code — not assumptions:

- **No database.** Every activity-log event and every admin/manager control (force-logout, project block) is stored in a plain in-memory JavaScript array/Map ( `globalThis.__hiranandaniActivity` , `globalThis.__hiranandaniControls` ) inside the API routes.
- **No real authentication.** There are exactly 5 hardcoded accounts in `src/lib/users.ts` , with **plaintext passwords committed in source code** (e.g. `admin123` , `staff123` ).
- **Sessions are just cookies.** Login sets a few plain cookies (role, name, email); nothing is signed or verified server-side. Anyone who can set a cookie can claim to be any role.
- **No rate limiting, no CSRF checks** on the endpoints that change state (login, force-logout, project block, activity logging).
- **Dashboards poll on fixed timers** (every 10–15 seconds) while a staff profile panel is open, on top of the main filtered query re-running on every filter change.

This is a working demo/prototype, not a backend built for concurrent daily production traffic yet. That is fine for what it was built for so far — but it is the reason "many staff, many logins a day" is a real risk today, not a hypothetical one.

## 2. Why this can break ("website fatega") under real multi-staff use

| Risk                                         | What actually happens                                                                                                                                                                                                                                                                                                    |
|----------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Data disappears on every deploy / cold start | In-memory storage lives only inside one running server process. Any redeploy, crash, or serverless cold start wipes the entire activity log and every force-logout/block setting instantly, with zero warning.                                                                                                           |
| Inconsistent data across staff               | Under real traffic, more than one server instance can run at once (common on serverless hosting). Each instance has its <i>own separate</i> in-memory store. A login logged by instance A can be invisible to a manager whose dashboard request lands on instance B — reports quietly go incomplete instead of erroring. |
| Unbounded memory growth                      | Every login, search, and click is appended forever with no archiving. Many staff × many logins a day × weeks of use means the in-memory array (and every full-history scan on it) keeps growing and getting slower — and it lives entirely in RAM.                                                                       |
| No password security                         | Plaintext passwords in source control, only 5 fixed accounts, no lockout after failed attempts.                                                                                                                                                                                                                          |
| No abuse protection                          | Anyone who finds <code>/api/activity</code> or <code>/api/controls</code> can POST directly with no login check, no rate limit, no origin check — they could flood the log or force-logout any staff member by email alone.                                                                                              |

|                                |                                                                                                                                                                                     |
|--------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Polling load multiplies</b> | Every open manager/admin panel independently polls every 10–15s. With several managers/admins watching at once, that's steady background load with no caching layer in front of it. |
|--------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

### 3. The full-proof plan, in priority order

#### P0 — do before real daily multi-staff use

1. **Move the activity log and controls store to a real database** (e.g. Postgres via a managed provider). This single change fixes the data-loss-on-restart problem, the cross-instance inconsistency problem, and the unbounded-memory problem all at once.
2. **Add indexes** on the fields every dashboard query filters/sorts by (staff email, manager email, timestamp) so lookups stay fast as history grows.
3. **Hash passwords** (bcrypt/argon2) instead of storing them in plaintext, or move login to a real managed auth provider.
4. **Make sessions server-verified** — a signed/http-only session token, not just "a cookie happens to be present."
5. **Add basic rate limiting** on login and on every state-changing POST endpoint, so one bug or bad actor can't flood the system or lock out staff.

#### P1 — soon after

1. Replace fixed-interval polling with less frequent polling, or only refresh while a panel is actually visible/focused.
2. Default the activity-log API to a bounded, paginated, date-limited query instead of always being able to return full history.
3. Add same-site/origin checks (CSRF protection) on every action that changes state.
4. Turn on basic monitoring/error alerting so a crash or traffic spike is visible before staff report it.

#### P2 — nice to have, not urgent

1. Automated backup of the activity log (CSV export already exists manually; automate it).
2. A load test simulating ~20–30 concurrent staff logins before a real go-live, to confirm the above actually holds up.

**Bottom line:** Everything in P0 is what stands between "works fine in a demo with 2–3 people" and "silently loses data or falls over with real staff logging in throughout the day." None of it is optional once this goes to more than a handful of real daily users.

## 4. What the dashboard does *not* need right now

Reviewing `SessionReports.tsx` for scope: several features are demo polish built ahead of any real usage data. None of these make the site more reliable — they're safe to leave as-is, simplify, or postpone until real managers ask for them, so time goes to Section 3 first.

| Feature                                                                                                    | Why it can wait                                                                                                                                                                    |
|------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Defer</b> Visual "roadmap" diagram (boxes + arrows) view of a walkthrough                               | The list/replay timeline view already shows the same data. Two visual modes for one dataset before anyone's asked for either is speculative.                                       |
| <b>Defer</b> Auto-playing "Replay" timer control                                                           | A nice demo trick; a manager reading a list is enough day-to-day.                                                                                                                  |
| <b>Defer</b> Three separate analytics charts (Busiest Hours, Content Breakdown, Top Properties) on day one | Meaningless until there are weeks of real data behind them. One combined summary is enough until volume justifies more.                                                            |
| <b>Simplify</b> Three-state staff status (Online / In Meeting / Busy)                                      | Heuristic-inferred nuance a manager likely doesn't need at a glance. Online/Offline covers the real question ("is this person working right now").                                 |
| <b>Simplify</b> Per-project block/unblock toggle for every project, for every staff member                 | A lot of UI for a control nobody has asked to use yet. A single "suspend this staff member" switch covers the real need; per-project nuance can come later if actually requested.  |
| <b>Defer</b> Custom hand-rolled date-range calendar picker                                                 | The three presets (Today / 7 days / 30 days / All time) already cover almost every real question a manager asks. The full calendar is extra surface area to maintain for rare use. |
| <b>Defer</b> Further navbar visual polish (glass effects, animations)                                      | Cosmetic only — has no effect on whether the site stays up under load. Should not take time away from Section 3's P0 items.                                                        |
| <b>Defer</b> Export formats beyond CSV (Excel, PDF, etc.)                                                  | CSV export already exists and covers the need; more formats are effort with no current demand.                                                                                     |

**Rule of thumb going forward:** only bring a cut/deferred feature back once a real manager or admin actually asks for it in daily use — not because it looks good in a demo.

## 5. Suggested order of work

| When    | Focus                                                                                                |
|---------|------------------------------------------------------------------------------------------------------|
| Week 1  | Move activity + controls data to a real database; hash passwords.                                    |
| Week 2  | Server-verified sessions; rate limiting & CSRF checks; paginate the activity API.                    |
| Week 3  | Load test with ~20+ simulated concurrent logins; add monitoring/alerts; hold off on Section 4 items. |
| Ongoing | Re-add any deferred dashboard feature only once real staff usage shows it's actually needed.         |

Generated from a direct review of the current codebase (no external assumptions). Re-run this review once the database migration lands, since several risks above are resolved by that single change.